

1 Synthetic Data Experiments

1.1 Rayleigh Quotient Problem

As requested by the reviewers, we have added comparisons between our proposed Riemannian Dueling Normalized Gradient Descent (RDNGD) and the Euclidean Dueling method with Projection, (PDNGD, Projected Dueling Normalized Gradient Descent). Unlike RDNGD, PDNGD is a purely heuristic method lacking theoretical guarantees. In our evaluations on the Rayleigh Quotient Problem across various cosine annealing stepsizes, RDNGD consistently achieves higher final accuracy. While PDNGD exhibits a marginal advantage in CPU time, RDNGD provides better precision. See Figure 1 and Figure 2.

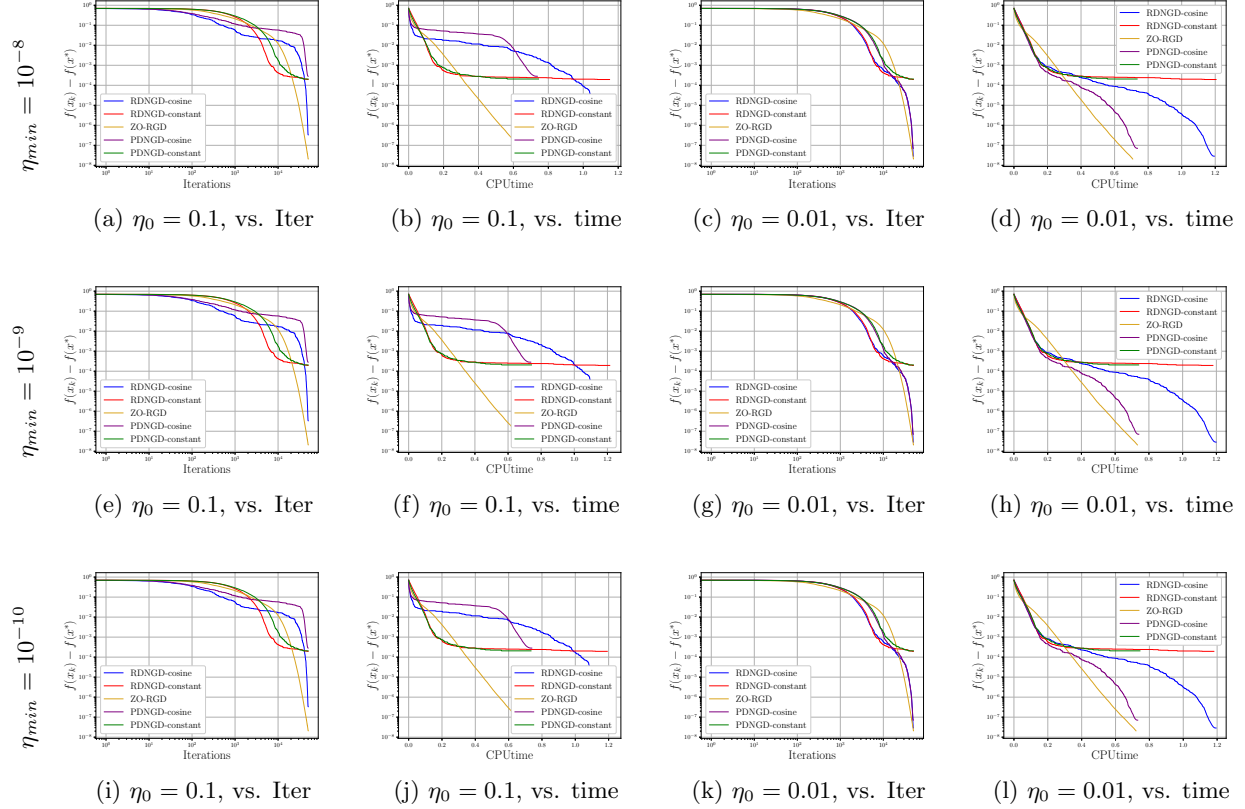


Figure 1: Comparisons between RDNGD and PDNGD on the Rayleigh quotient maximization problem ($d = 100$). The three rows correspond to minimum step sizes $\eta_{min} = 10^{-8}, 10^{-9}$, and 10^{-10} , respectively. For each η_{min} , we evaluate two initial step sizes ($\eta_0 \in \{0.1, 0.01\}$) and plot the loss against both iteration and CPU time.

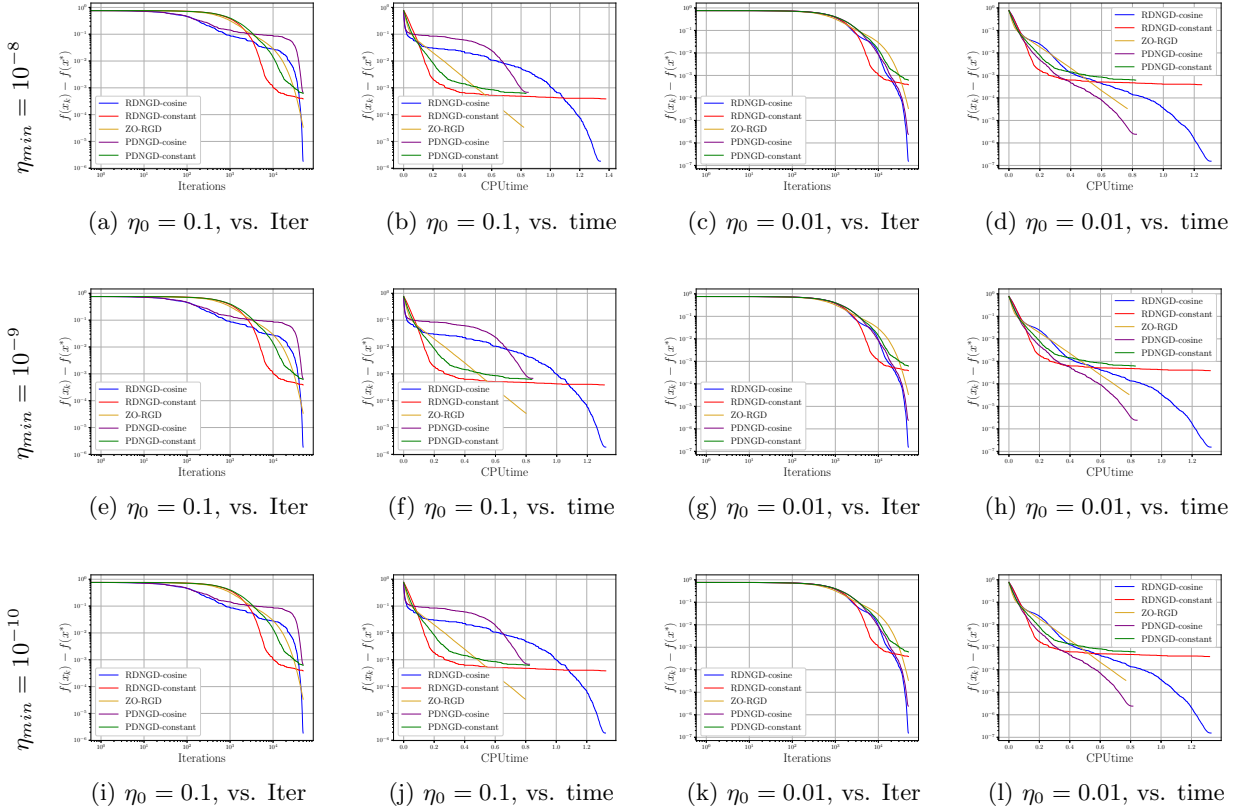


Figure 2: Comparisons between RDNGD and PDNGD on the Rayleigh quotient maximization problem ($d = 150$). The three rows correspond to minimum step sizes $\eta_{min} = 10^{-8}, 10^{-9}$, and 10^{-10} , respectively. For each η_{min} , we evaluate two initial step sizes ($\eta_0 \in \{0.1, 0.01\}$) and plot the loss against both iteration and CPU time.

1.2 Karcher Mean Problem

We conducted a second set of experiments on the Karcher Mean Problem over the Symmetric Positive Definite (SPD) manifold. Consistent with the previous results, we compared RDNGD against the heuristic PDNGD. Beyond its lack of theoretical guarantees, PDNGD encounters fundamental projection challenges on the SPD manifold. A direct Euclidean dueling step yields an intermediate matrix $A = X + U$ that is not necessarily symmetric. More importantly, since the SPD manifold is an open set, projection is inherently ill-defined when A contains negative eigenvalues. While the closest positive semidefinite matrix can be found via $Y = \frac{A+A^T}{2}$ with negative eigenvalues clipped to zero, the SPD manifold strictly requires strictly positive eigenvalues. To implementing PDNGD, we symmetrize the matrix and clip any non-positive eigenvalues to a strictly positive hyperparameter (set to $1e-8$ in our experiments).

RDNGD-cosine yields better precision and efficiency than PDNGD-cosine across all configurations, as shown in Figures 3 and 4.

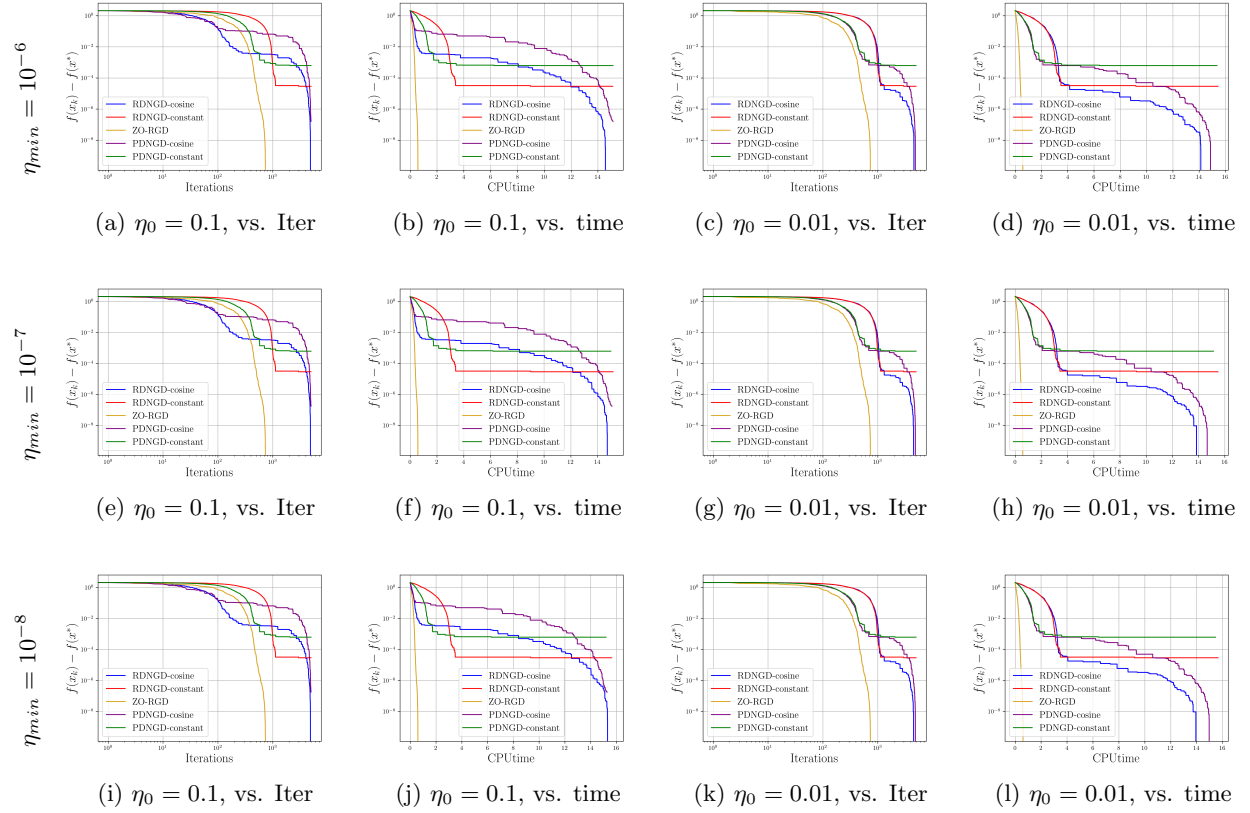


Figure 3: Comparisons between RDNGD and PDNGD on the Karcher Mean Problem over the SPD manifold ($d = 5$). The three rows correspond to minimum step sizes $\eta_{min} = 10^{-6}, 10^{-7}$, and 10^{-8} , respectively. For each η_{min} , we evaluate two initial step sizes ($\eta_0 \in \{0.1, 0.01\}$) and plot the loss against both iteration and CPU time.

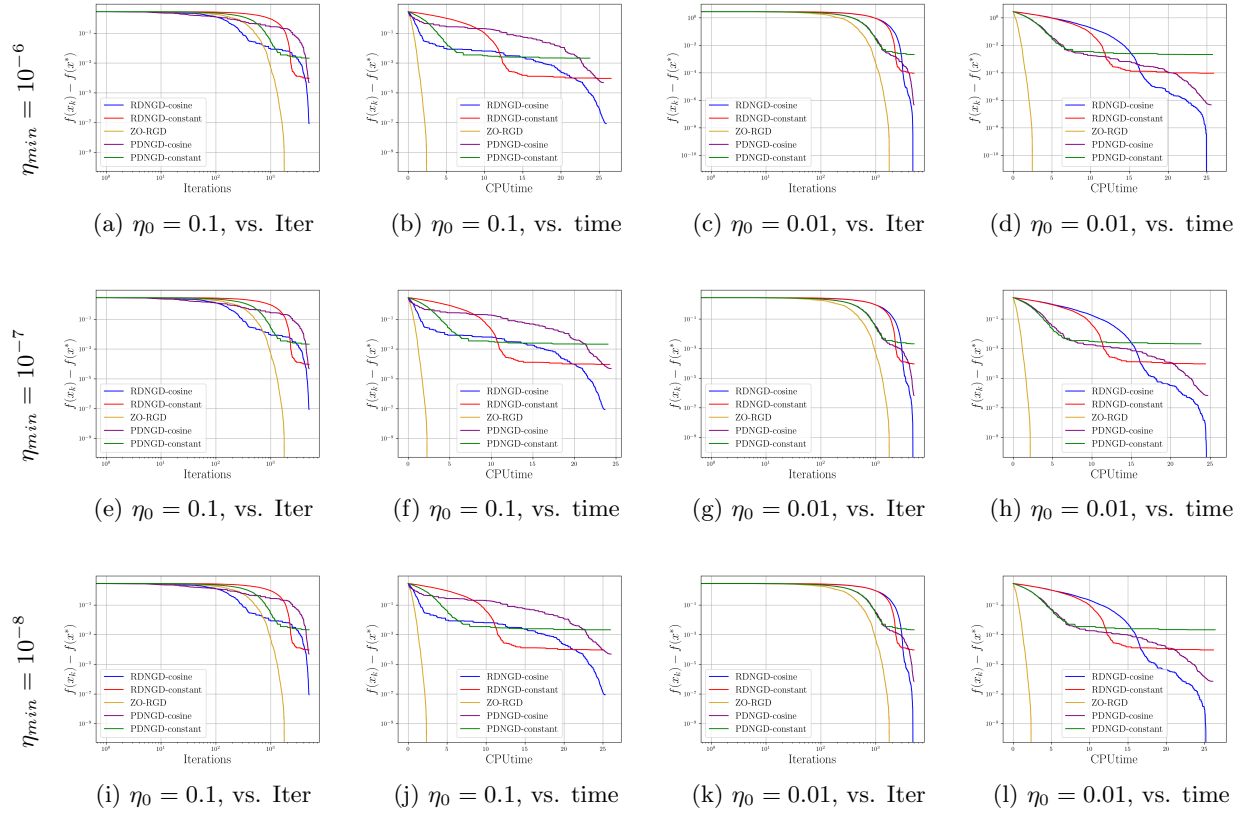


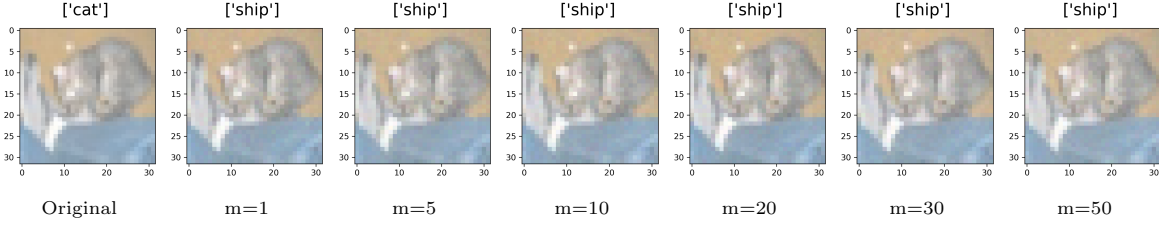
Figure 4: Comparisons between RDNGD and PDNGD on the Karcher Mean Problem over the SPD manifold ($d = 10$). The three rows correspond to minimum step sizes $\eta_{min} = 10^{-6}, 10^{-7}$, and 10^{-8} , respectively. For each η_{min} , we evaluate two initial step sizes ($\eta_0 \in \{0.1, 0.01\}$) and plot the loss against both iteration and CPU time.

2 Attack on DNNs

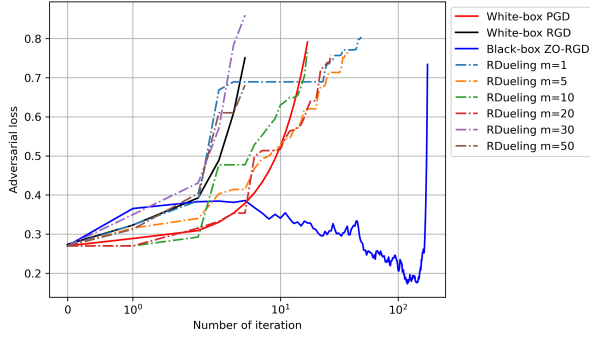
In this experiment, we do an ablation study of the batchsize m , and we also compare our results with the Euclidean Dueling with Projection (PDNGD) and ZO-ADMM.

2.1 Abalation study of perturbation batchsize m

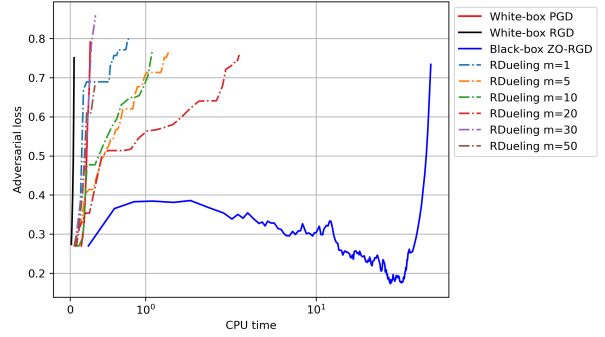
We conduct an ablation study on the perturbation batch size $m \in \{1, 5, 10, 20, 30, 50\}$. While $m = 1$ successfully attacks one of the three examples, it reaches the maximum iteration limit in other cases despite a steady increase in loss, and thus does not successfully attack the image within the given iteration budget. Generally, a larger m reduces the total number of iterations but increases the computational cost per iteration. As shown in Figures 5–7, $m \in \{5, 10\}$ yields the optimal CPU time.



(a) Generated adversarial images



(b) Adversarial Loss vs. Iteration

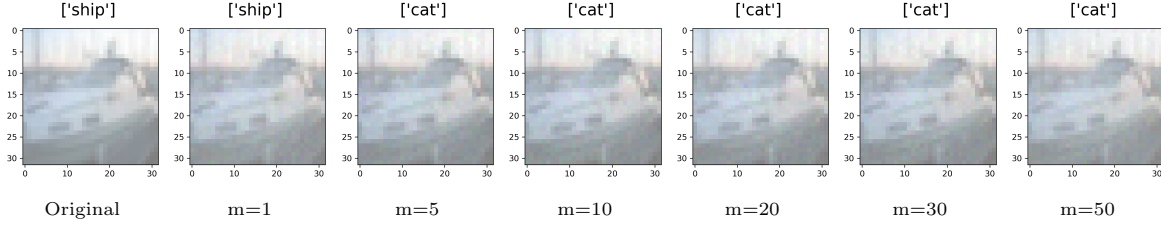


(c) Adversarial Loss vs. Time

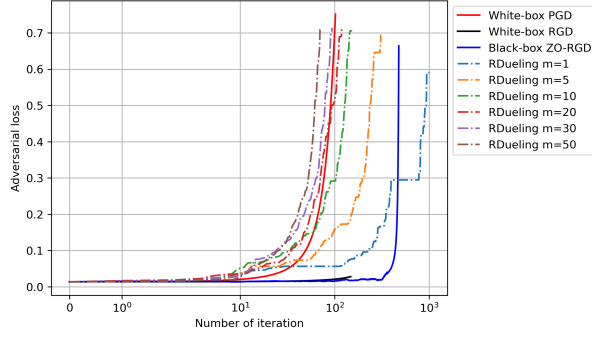
Figure 5: Additional attack result on CIFAR-10 (Example 1).

2.2 Comparison with PDNGD and ZO-ADMM

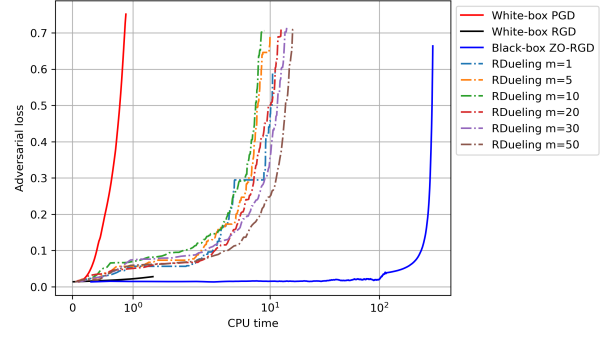
We compare our proposed RDNGD with PDNGD and ZO-ADMM. As shown in Figures 8–10, RDNGD achieves similar precision and CPU time to PDNGD. While ZO-ADMM (implemented with a sample size of 10) outperforms RDNGD in certain cases, we emphasize that RDNGD only requires a comparison oracle, whereas ZO-ADMM relies on explicit function values, and hence not applicable when only comparison oracle is available. See Figures 8–10.



(a) Generated adversarial images

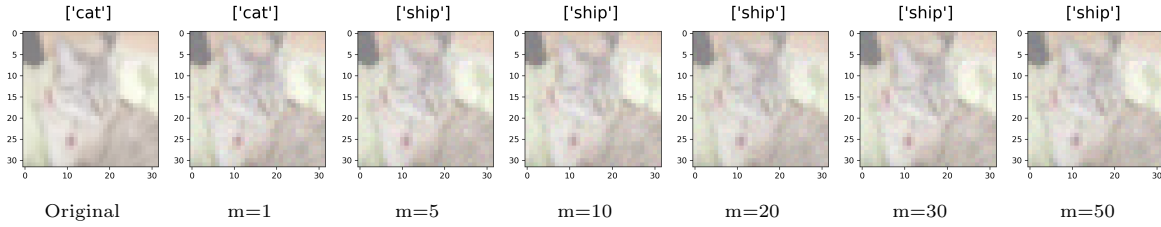


(b) Adversarial Loss vs. Iteration

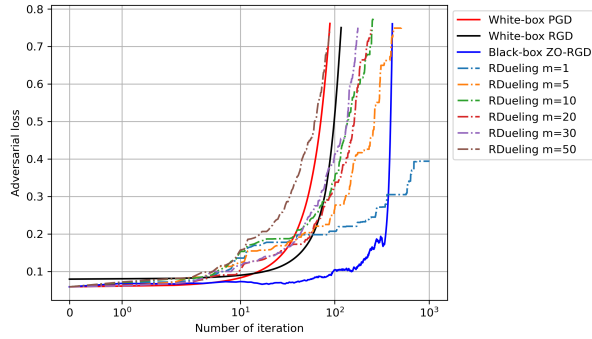


(c) Adversarial Loss vs. Time

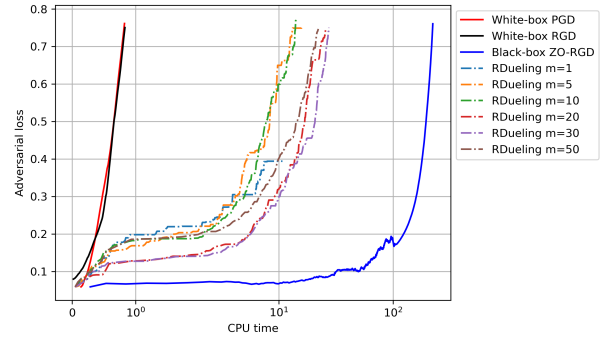
Figure 6: Additional attack result on CIFAR-10 (Example 2).



(a) Generated adversarial images

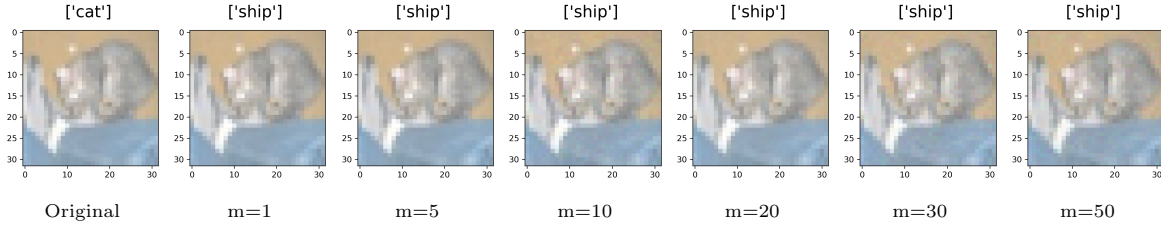


(b) Adversarial Loss vs. Iteration

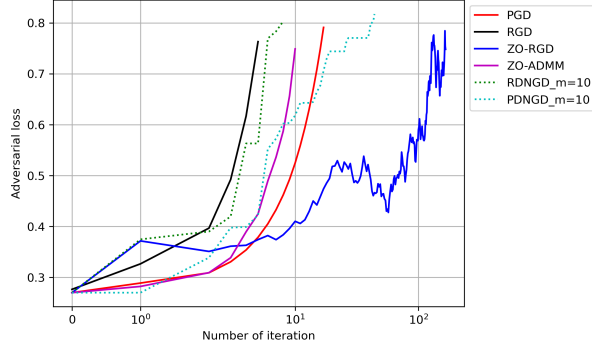


(c) Adversarial Loss vs. Time

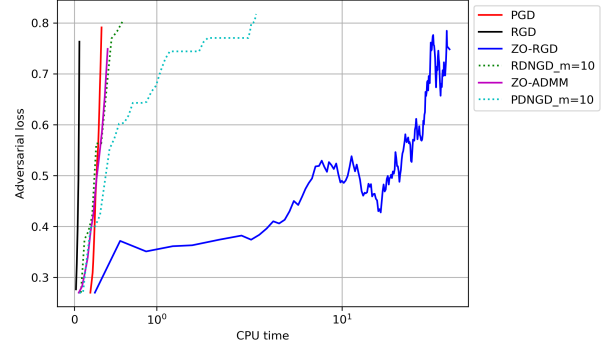
Figure 7: Additional attack result on CIFAR-10 (Example 3).



(a) Generated adversarial images

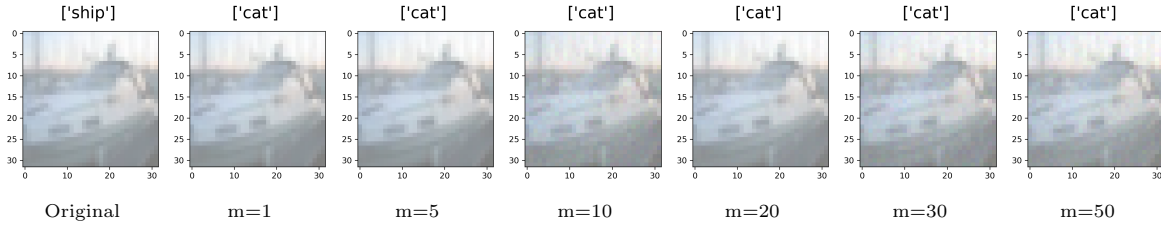


(b) Adversarial Loss vs. Iteration

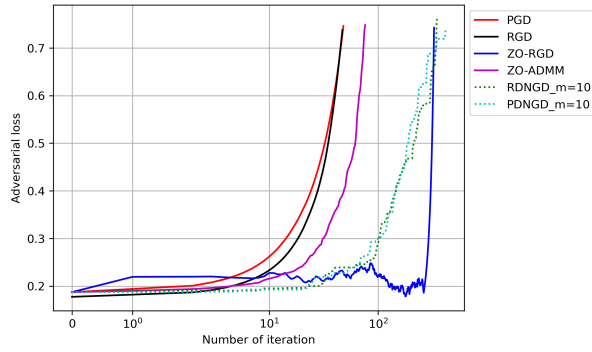


(c) Adversarial Loss vs. Time

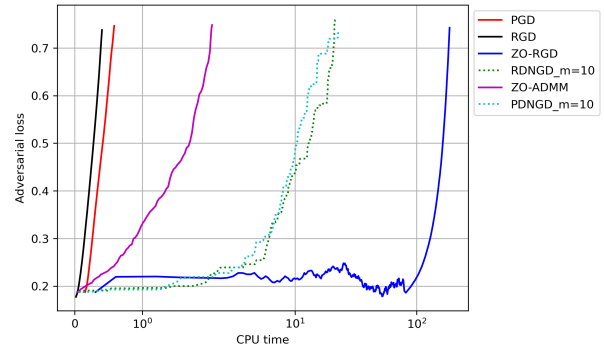
Figure 8: Additional attack result on CIFAR-10 (Example 1).



(a) Generated adversarial images

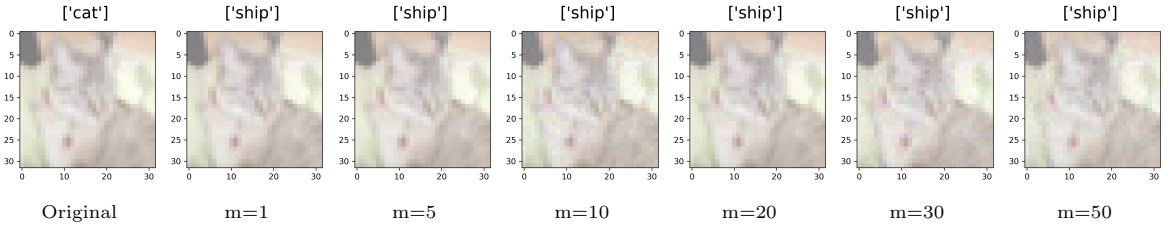


(b) Adversarial Loss vs. Iteration

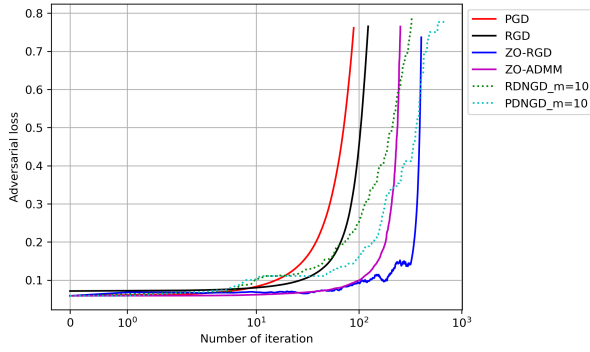


(c) Adversarial Loss vs. Time

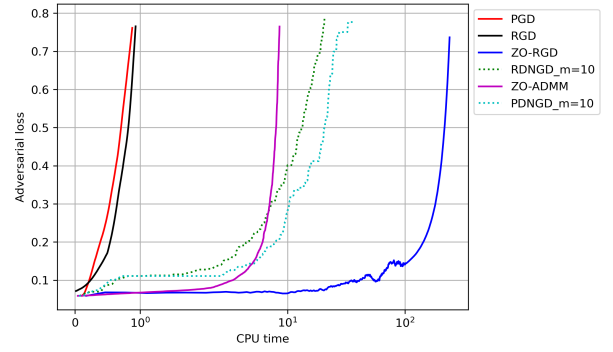
Figure 9: Additional attack result on CIFAR-10 (Example 2).



(a) Generated adversarial images



(b) Adversarial Loss vs. Iteration



(c) Adversarial Loss vs. Time

Figure 10: Additional attack result on CIFAR-10 (Example 3).